

Examen de programmation

Durée: 2h

Version: **A**

Instructions

- Répondez sur la feuille de réponse séparée **à votre nom**. **Aucune** réponse écrite sur le présent énoncé ne sera prise en compte dans la correction
- Répondez en noircissant la case de la réponse avec un **stylo ou un feutre** (pas de crayon).
- Noircissez aussi la version (**A** ou **B**) de votre énoncé sur la feuille de réponse.
- Pour corriger une réponse, **effacez complètement** la mauvaise case avec un typex avant d'en noircir une autre.
- Vous ne devez pas remettre l'énoncé. Vous pouvez donc l'utiliser comme brouillon.
- Il n'y a pas de points négatifs.
- **Aucun** document n'est autorisé.

Question 1 (1 point)

Laquelle des propositions suivantes est fausse ?

- A) En Python, on peut multiplier une liste par un entier
- B) Un programme est une suite d'instructions visant à la résolution d'un problème.
- C) La mémoire RAM est volatile.
- D) Une variable globale est définie en dehors de toute fonction
- E) Un SSD est un stockage de masse
- F) Un pseudo-code n'est écrit dans aucun langage particulier.
- G) Desktop\main.py est un chemin relatif
- H) Le type `complex` existe en Python
- I) Le stockage de masse est généralement plus grand que la mémoire
- J) La commande `pwd` permet de lister le contenu du répertoire courant

Question 2 (1 point)

Que vaut `y` à la fin de l'exécution du code suivant ?

```
1 import numpy as np
2
3 x = np.array([1, 2])
4 y = np.array([4, 2])
5 y = x ** y
6 y += 1
```

- A) `[1 2 4 2 1]`
- B) `[6 6]`
- C) `[2 5]`
- D) `[[1 4] [2 2]]`
- E) `8`
- F) `[1 1 1 1 2 2 1]`
- G) `[[1 4] [2 2] [1]]`
- H) `[4 2 2 1]`
- I) `None`
- J) Message d'erreur

Question 3 (1 point)

Que vaut **b** à la fin de l'exécution du code suivant ?

```
1  b = 9999999
2  for x in [2, 5, 3]:
3      if x < b:
4          b = x
```

- A) 10 B) 2 C) 1.5 D) 5 E) 1
- F) 253 G) "253" H) 3 I) 30 J) Message d'erreur

Question 4 (2 points)

Que va afficher le code suivant ?

```
1  def fun(L1, L2):
2      res = []
3      for i in range(len(L1)):
4          d = [L1[i], L2[i]]
5          res += d
6      return res
7
8
9  print(fun([4, 6, 7], [-1, 3, 10]))
```

- A) [3, 9, 17] B) [-1, 3, 10]
- C) [5, 3, 3] D) [17, 12]
- E) [] F) [4, -1, 6, 3, 7, 10]
- G) 29 H) [[4, -1], [6, 3], [7, 10]]
- I) Message d'erreur J) Aucune des propositions

Question 5 (2 points)

Que va afficher l'exécution du code suivant ?

```
1 L = [2, 5, 7, 3, 1]
2 x = 0
3 r = []
4 for a in L:
5     x += str(a)
6     r = r + [int(x)]
7 print(r)
```

- A) ['2', '5', '7', '3', '1']
- B) ['25731']
- C) ['2', '25', '257', '2573', '25731']
- D) []
- E) '225257257325731'
- F) [2, 25, 257, 2573, 25731]
- G) [[2], [25], [257], [2573], [25731]]
- H) Le programme ne s'arrête pas (boucle infinie)
- I) Le programme plante (message d'erreur)

Question 6 (3 points)

Dans le code suivant, la fonction `smooth(L)` retourne une nouvelle liste dans laquelle chaque élément est remplacé par la moyenne arithmétique de lui-même, de son prédécesseur et de son successeur dans la liste d'origine. Pour les extrémités, on considère que les éléments en dehors de la liste valent 0. Malheureusement, le code plante lorsqu'on l'exécute.

```
1 def get_value(L, i, default):
2     if i < 0 or i >= len(L):
3         return default
4     else:
5         return L[i]
6
7 def smooth(L):
8     res = []
9     for i in range(len(L)):
10        elem = 0
11        for di in [-1, 0, 1]:
12            elem += get_value(L, i + di)
13        res.append(elem / 3)
14    return res
15
16 print(smooth([2, 1, 6, 2, 1]))
17 # devrait afficher
18 # [1.0, 3.0, 3.0, 3.0, 1.0]
19 # car
20 # [(0+2+1)/3, (2+1+6)/3, (1+6+2)/3, (6+2+1)/3, (2+1+0)/3]
```

Le message d'erreur est le suivant :

```
Traceback (most recent call last):
  File "/Users/lur/exam.py", line 16, in <module>
    print(smooth([2, 1, 6, 2, 1]))
    ~~~~~^~~~~~
  File "/Users/lur/exam.py", line 12, in smooth
    elem += get_value(L, i + di)
             ~~~~~^~~~~~
TypeError: get_value() missing 1 required positional argument: 'default'
```

Que faut-il changer pour corriger la fonction ? (les $\rightarrow$ indiquent le niveau d'indentation)

- A) ligne 1: `def get_value(L, i, default=0):`
- B) ligne 2: $\rightarrow$ `if i < 0 and i >= len(L):`
- C) ligne 3: $\rightarrow$ `return None`
- D) ligne 8: $\rightarrow$ `res = L`
- E) ligne 10: $\rightarrow$ `elem = L[i]`
- F) lignes 12: $\rightarrow$ `elem += get_value(L, i + di, di)`
- G) Les propositions A et C simultanément

Question 7 (4 points)

Dans le programme suivant, la fonction `find_words_with_n_vowels()` devrait renvoyer les mots de la liste reçue en paramètre contenant exactement `n` voyelles :

```
1  def count_vowels(word):
2      res = 0
3      for c in word:
4          if c == "aeiouy":
5              res += 1
6      return res
7
8  def find_words_with_n_vowels(words, n):
9      res = []
10     for word in words:
11         count = count_vowels(word)
12         if count == n:
13             res.append(word)
14     return res
15
16 print(find_words_with_n_vowels(["banane", "pomme", "groseille", "cerise",
    "fraise"], 3))
```

Le résultat du programme devrait être le suivant:

```
['banane', 'cerise', 'fraise']
```

Cependant une erreur s'est glissée dans le programme et on obtient ceci :

```
[]
```

Que faut-il changer pour corriger la fonction ? (les → indiquent le niveau d'indentation)

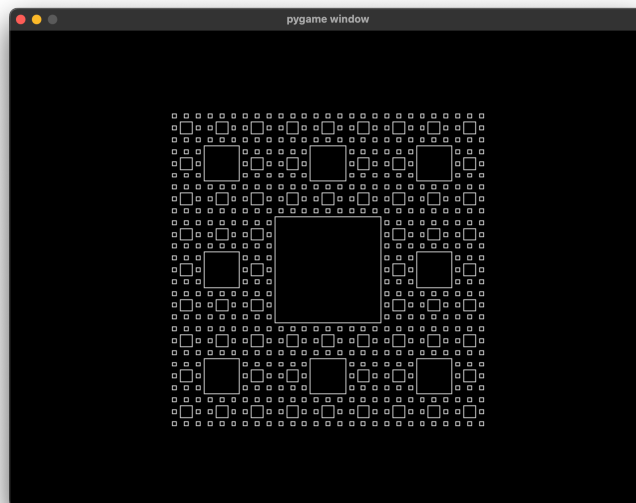
- A) Ligne 2: → `res = word[0]`
- B) Ligne 2: → `res = ""`
- C) Ligne 4: → → `if c in "aeiouy":`
- D) Ligne 5: → → → `res += c`
- E) Ligne 9: → `res = None`
- F) Ligne 9: → `res = 0`
- G) Ligne 11: → → `count = count_vowels(words)`
- H) Ligne 12: → → `if len(word) == n:`
- I) Ligne 13: → → → `res += word`

Carpette de Sierpiński

Voici le code d'un programme pygame où il manque trois instructions :

```
1  # première instruction manquante
2
3  pygame.init()
4
5  def square(screen, x, y, side):
6      pygame.draw.line(screen, (255, 255, 255), (x, y), (x + side, y))
7      pygame.draw.line(screen, (255, 255, 255), (x + side, y), (x + side, y + side))
8      pygame.draw.line(screen, (255, 255, 255), (x + side, y + side), (x, y + side))
9      # deuxième instruction manquante
10
11  def carpet(screen, x, y, side):
12      third = side / 3
13      if third > 2:
14          for i in range(3):
15              for j in range(3):
16                  if i == 1 and j == 1:
17                      square(screen, x + i * third, y + j * third, third)
18                  else:
19                      carpet(screen, x + i * third, y + j * third, third)
20
21  size = (800, 600)
22  # troisième instruction manquante
23
24  while not pygame.event.peek(pygame.QUIT):
25      pygame.draw.rect(screen, (0, 0, 0), pygame.Rect((0, 0), size))
26      carpet(screen, 200, 100, 400)
27      pygame.display.flip()
```

Ce programme devrait afficher la carpedette de Sierpiński. Cette figure se construit en divisant un carré en neuf (3 divisions en largeur et en hauteur); on trace le carré du centre et on ré-applique la même procédure sur les 8 autres carrés:



Question 8 (2 points)

Dans le programme de la carpette de Sierpiński ci-dessus, quelle proposition choisissez-vous pour la première instruction manquante ? (*Cette instruction n'est pas indentée*)

- A) `import turtle`
- B) `from pygame import goto, forward, left, done, up, down`
- C) `from turtle import goto, forward, right, done, up, down`
- D) `from turtle`
- E) `from turtle import goto, forward, left, done, up, down`
- F) `from turtle import goto, forward, left, up, down`
- G) `import pygame`
- H) `import goto, forward, left, done, up, down`

Question 9 (2 points)

Dans le programme de la carpette de Sierpiński ci-dessus, quelle proposition choisissez-vous pour la deuxième instruction manquante ? (*Cette instruction est indentée 1 fois*)

- A) `pygame.draw.line(screen, (255, 255, 255), (x + side, y), (x, y))`
- B) `pygame.draw.line(screen, (255, 255, 255), (x, y + side), (x, y))`
- C) `pygame.draw.line(screen, (255, 255, 255), (x, y + side), (x + side, y))`
- D) `pygame.draw.line(screen, (255, 255, 255), (x, y + side), (x, y + side))`
- E) `pygame.draw.line(screen, (255, 255, 255), (x, y + side), (x + side, y + side))`
- F) `pygame.draw.line(screen, (255, 255, 255), (x + side, y + side), (x, y))`
- G) `pygame.draw.line(screen, (255, 255, 255), (x + side, y + side), (side, side))`
- H) `pygame.draw.line(screen, (255, 255, 255), (side, y + side), (x, y))`
- I) `pygame.draw.line(screen, (255, 255, 255), (x, y), (side, y))`
- J) `pygame.draw.line(screen, (255, 255, 255), (x + side, y), (x, side))`

Question 10 (2 points)

Dans le programme de la carpette de Sierpiński ci-dessus, quelle proposition choisissez-vous pour la troisième instruction manquante ? (*Cette instruction n'est pas indentée*)

- A) `width = size[0]`
- B) `title = "Carpette de Sierpiński"`
- C) `pygame.display.set_mode(size)`
- D) `points = []`
- E) `clock.tick(60)`
- F) `res = 0`
- G) `color = (255, 255, 255)`
- H) `screen = pygame.display.set_mode(size)`
- I) `clock = pygame.time.Clock()`
- J) Aucune des propositions

Brouillon